

CUDA_ERROR_UNKNOWN GDN Kernels

Thien Tran Simon Veitner Yue Zhang Mayank Agarwal Chunan Zeng

April 30, 2026

1 Introduction

Gated Delta Networks (GDN) provide an efficient alternative to attention for long-context sequence modeling, but achieving high performance on GPUs requires careful kernel design due to their mix of dense linear algebra, recurrence, and irregular workloads. In this work, we present Blackwell-optimized CUDA kernels for both decode and prefill regimes, focusing on Tensor Core utilization, memory efficiency, and load balancing.

2 Decode

Decoding is straightforward: it computes the new state given the current state. We implemented and/or tuned two versions for different workloads: `gdn_decode_small_batch.cu` and `gdn_decode_kernel_large_batch.py`.

2.1 `gdn_decode_small_batch.cu` Details

For small batch, each CTA reads directly from GMEM into registers, since it only processes one V tile.

Tiling and parallelism: Grid is $B \times \text{kNumVHeads} \times \text{kNumVTiles}$ with $\text{kNumVTiles} = 128/4 = 32$ and 128 threads per block (4 warps $\times$ 32 threads). Each warp maps to one **V row** within a 4-row tile. This lines up the compute with a fixed head size of 128.

Vectorized Q/K loads: `LoadBf16x4GlobalNc` issues a single `ld.global.nc` vector load of 4 bfloat16s per lane, converts to float in-register, and uses `L1::evict_first` to steer cache behavior.

State update and output: It keeps per-lane `float4` state, uses `fmaf` for the fused multiply-add along K, uses `WarpAllReduceSum` for the dot products, and only lane 0 writes the bf16 output for that V row.

Stores to new_state: `st.relaxed.cta.global` with `L1::no_allocate` on a `float4` store (`StoreF32x4-RelaxedNoAllocate`) to cut L1 traffic on write-through/fresh state.

2.2 `gdn_decode_kernel_large_batch.py` Details

For large batches where each CTA processes multiple V tiles, we adopt the FlashInfer baseline CuteDSL kernel¹ which uses SMEM with `cp.async` (via `cute.copy()`) and ping-pong scheduling to hide memory access latency.

Vectorized Q/K/V loads: Similar to the small-batch kernel, it uses vectorized loads via `cute.autovec_copy()` are used for Q, K, and V.

Pipelined loads for state tiles: A 2-stage ping-pong style pipeline that uses `cp.async` to prefetch state tiles into shared memory while the math runs on the current stage.

Batch-dependent config tuning: `num_blocks_per_state`: **16** for small batch size, **8** for $B \geq 16$, **4** for $B \geq 48$ (fewer, fatter blocks as batch grows to amortize launch and scheduling). `tile_v`: **4** when $B \leq 8$, else **8**, coarser V-tiling when the kernel has enough work to use it well.

¹https://github.com/flashinfer-ai/flashinfer/blob/v0.6.7/flashinfer/gdn_kernels/gdn_decode_pretranspose.py

3 Prefill

3.1 Background

For Prefill workloads, a **chunked implementation** is preferred to better utilize Tensor Cores. The original FLA reference² uses five kernels: `chunk_scaled_dot_kkt_fwd()`, `solve_tril()`, `recompute_w_u_fwd()`, `chunk_gated_delta_rule_fwd_h()`, and `chunk_fwd_o()` (excluding α, β computation and metadata preparation), while the FlashInfer baseline³ uses a fully fused kernel. Although fused kernels are typically expected to be faster, FLA performs better on competition workloads due to **load balancing**. In a fused design, each threadblock handles one sequence head, so uneven sequence lengths lead to idle blocks and poor GPU utilization. In contrast, FLA processes chunks independently, distributing work more evenly across threadblocks.

Motivated by this, we follow FLA and use four kernels: (1) Metadata preparation, (2) `kkt`, inverse, and UW kernel, (3) State update H kernel, and (4) Output O kernel.

3.2 Kernel Design

3.2.1 Metadata preparation

We compute the chunk metadata in a device-side kernel rather than on the host (as FLA does via `cu_seqlens.tolist()`) to avoid the device-to-host synchronization. Downstream kernels read the total chunk count from GPU memory at runtime.

3.2.2 `kkt`, inverse, and UW kernel

This kernel is a fusion of three FLA kernels: `chunk_scaled_dot_kkt_fwd()`, `solve_tril()`, and `recompute_w_u_fwd()`. For one chunk and one V -head, the kernel takes K, V , the per-token gate inputs a, b , and the per-head scalars $A_{\log}$ and dt_{bias} , and produces the cumulative gate g_{cu} together with the two outputs U and W (full equations in Appendix B.1). g_{cu} is used by downstream H and O kernels. β_t is used only in this kernel, so it does not need to be written to global memory. We also save global memory traffic for intermediates like $KK^\top$ and $(I + A)^{-1}$.

Matrix inverse with MMA. Most existing GDN kernels compute $(I + A)^{-1}$ using forward substitution, which is slow and does not utilize Tensor Cores. We consider two matmul-based approaches: **Neumann series** and **Newton–Schulz iteration** (see Appendix C for details). Based on our experiments, Newton–Schulz iteration is slightly faster and more accurate when used with BF16 MMA. Hence, we adopt Newton–Schulz in our final kernel. Following FLA, we compute the direct inverse for 16×16 diagonal tiles, then compute the inverse of off-diagonal tiles via block forward-substitution.

Implementation. We adopt a warp-specialized, persistent kernel design. There are 10 warps with specialized roles, as shown in Table 1.

Warps (Role)	Description
Warp 9 (TMA)	Issues TMA loads for K and V from gmem to smem.
Warp 8 (MMA)	Issues <code>tcgen05.mma</code> for KKT (MMA #1).
Warps 4–7 (INV)	Prepare β/g_{cu} , mask KKT into A , invert $(I + A)$, and prepare inputs for the U/W MMA.
Warp 8 (MMA)	Issues <code>tcgen05.mma</code> for UW (MMA #2).
Warps 0–3 (EPI)	Drain UW from tmem to smem and issue TMA stores to gmem.

Table 1: Logical flow of the `kkt`+inverse+UW kernel.

For matrix inversion, we use BF16 `mma.sync.aligned.m16n8k16` to match the 16×16 matmul tiles involved. MMA #2 uses a tmem input for the A operand to achieve low latency.

²https://github.com/fla-org/flash-linear-attention/blob/v0.5.0/fla/ops/delta_rule/chunk.py#L18

³https://github.com/flashinfer-ai/flashinfer/blob/v0.6.9/flashinfer/gdn_kernels/blackwell/gated_delta_net_chunked.py

Areas for improvement. Inverse warps currently stall the MMA warp: after issuing the KKT MMA, the MMA warp has to wait for the inverse warps’ results before issuing the UW MMA. To better overlap Tensor Cores with CUDA cores, we can apply a ping-pong schedule similar to FA4’s design (compute two chunks at the same time, so that MMA can issue KKT for the 2nd chunk while CUDA cores perform inversion for the 1st chunk).

3.2.3 State update H kernel

This is a direct port of FLA’s `chunk_gated_delta_rule_fwd_h()` to CUDA C++. Each threadblock is responsible for one head of a sequence. We iterate over the chunks in that sequence, compute the V delta, and update the state matrix H (full equations in Appendix B.2). Intermediate H and V_{new} are stored to gmem for the subsequent O kernel.

MMA #1 data layout ($V_{new} = U - WH^\top$). H is loaded from the tmem written by the previous MMA #2. Hence, it is natural to use H as the tmem input for MMA #1. To do so, we swap the input operands, making H the A operand, effectively computing $V_{new}^\top = U^\top - HW^\top$ (transposed V_{new}).

MMA #2 data layout ($H_{new} = \tilde{H} + \tilde{V}_{new}^\top K$). $V_{new}^\top$ from MMA #1 needs to be transposed back to row-major layout for the O kernel. We pair `tcgen05.ld.16x256b` (tmem $\rightarrow$ rmem) with `stmatrix.trans` (rmem $\rightarrow$ smem), and issue a TMA store (smem $\rightarrow$ gmem). Since V_{new} is already in transposed layout, it is also straightforward to store it back to tmem as the MMA #2 input (after applying scaling and converting to BF16).

Keep H in tmem. We initially kept the full H tensor in rmem throughout the kernel’s lifetime. MMA #2 performs $\tilde{V}_{new}^\top K$, and CUDA cores use FMA to compute H_{new} in-place. With `head_dim=128`, H has shape $[128, 128]$, and thus each thread across 4 warps consumes 128 registers just to hold H . To remove register pressure, we keep the full H tensor in tmem, and use it as the MMA #2 accumulator directly. All other operations can be done in chunks to keep register usage low.

U as MMA #1 accumulator. Inspired by $\tilde{H}$ being the MMA #2 accumulator, we also let U be the tmem accumulator for MMA #1. This reduces FMA pressure (subtraction is now done by the MMA), and creates a symmetric design for warpgroups handling H and V (Table 3). The full warp-specialized pipeline of the H kernel is shown in Table 4.

Areas for improvement. We do not implement a persistent kernel due to limited time. Since sequences in a batch can vary greatly in length, a persistent H kernel requires dynamic scheduling, which is supported natively by Blackwell’s Cluster Launch Control (CLC), or can be implemented in software (e.g., `atomicAdd()` on a counter in gmem).

3.2.4 Output O kernel

For each output head h_v , let H_v and $H_{q/k}$ denote the numbers of output heads and Q/K heads. The Q/K head used by h_v is

$$h_{q/k} = \left\lfloor \frac{h_v}{H_v/H_{q/k}} \right\rfloor.$$

For one chunk, $Q, K \in \mathbb{R}^{B_T \times K}$ are taken from Q/K head $h_{q/k}$, while the transformed values $V' \in \mathbb{R}^{B_T \times V}$, the chunk state $S \in \mathbb{R}^{V \times K}$, and the cumulative gate $g \in \mathbb{R}^{B_T}$ are specific to output head h_v . The per-head output is

$$O = \text{scale}(\text{diag}(\exp(g))QS^\top + AV'), \quad A_{ij} = \mathbf{1}_{j \leq i}(Q_i \cdot K_j) \exp(g_i - g_j).$$

The Flash Linear Attention baseline maps one program to (chunk, h_v). In our competition setting, $H_v = 8$ and $H_{q/k} = 4$, so two output heads share the same Q/K head. The baseline therefore forms the same raw $QK^\top$ tile twice, once for each output head in the pair.

Our CUDA kernel maps a CTA to (v_{tile} , chunk stream, $h_{q/k}$). The CTA owns the two output heads attached to one Q/K head, forms the raw $QK^\top$ tile once, and reuses it for both heads. Gate-dependent attention, $QH = QS^\top$, $OV = AV'$, and the final output are still computed independently for each output head; only the raw $QK^\top$ tile is shared. The CTA walks chunks with a strided `global_chunk_id += gridDim.y` loop, preserving

chunk-level load balancing for variable-length packed batches. With `BLOCK_V` = $V = 128$ in the competition configuration, a CTA covers the full value dimension for its output-head pair.

The implementation is warp-specialized. One TMA warp streams Q/K , V' , and S into shared memory; one MMA warp, through an elected lane, issues the $QK^\top$, QH , and attention-times-value Tensor Core products; four CUDA warps compute the centered gate factors, materialize the per-head lower-triangular attention tiles, and fuse the final gate/scale epilogue with stores. Appendix E gives the centered-gate form, the Tensor Memory capacity constraints that force the shared OUT bank and QH1 alias, and the synchronization/lifetime rules that make the alias safe.

3.3 Ensemble

Since the chunk-parallel pipeline’s fixed overhead (four kernel launches, metadata, padded intermediate tensors) dominates on very short inputs, we dispatch tiny workloads to a token-by-token recurrent CUDA kernel and route all other workloads to the chunk-parallel pipeline.

3.4 Optimization Graph

Optimization	Speedup
Baseline (FlashInfer CuteDSL)	1.00×
Initial Triton port following FLA’s 5-kernel structure	1.27×
Triton tuning (block sizes, autotune, fused gates)	2.01×
Replace KKT kernel with hand-written CUDA	2.12×
Improved inverse kernel (block-wise W/U + bf16 roundtrip + TF32)	2.24×
Fused O-kernel; remove host <code>.item()</code> sync	3.42×
Replace H kernel with CUDA implementation	3.74×
Dispatch CUDA-H for large N , Triton-H for small N	4.16×
H kernel: better warp specialization, TMA tuning	4.28×
<code>prep_meta</code> : device-side parallel scan (eliminate host sync)	4.33×
Replace Triton O kernel with CUDA O kernel (Q/K reuse across paired output heads)	4.48×
BF16 storage for A gmem intermediate	4.57×
CUDA inverse via Newton–Schulz on Tensor Cores (replaces forward-substitution)	4.94×
Fuse KKT + inverse + UW into one kernel; eliminate A gmem round-trip	5.49×
Tighten <code>elect_sync</code> scope to wrap only MMA-issue instructions	5.57×
Dispatch tiny workloads to recurrent CUDA kernel (final ensemble)	5.78×

Table 2: Prefill optimization progression on competition workloads, reported as cumulative end-to-end speedup over the FlashInfer CuteDSL baseline.

4 Conclusion

For both decode and prefill, we ship two complementary CUDA implementations per track and dispatch between them based on workload shape. Decode pairs a register-centric small-batch kernel with a CuTe-DSL ping-pong kernel tuned for large batches. Prefill pairs a four-kernel chunk-parallel pipeline that exploits Tensor Cores on medium and large inputs with a lightweight recurrent kernel for the tiny tail. Picking the right path per workload keeps both ends of the input distribution fast, and the resulting solution runs end-to-end on a single CUDA stream with no host-side synchronization.

The kernel designs in this work are predominantly human-led, while AI coding agents accelerated the bulk of the implementation, hyperparameter tuning, debugging, and the persistent exploration of numerical-precision fixes in the inverse kernel that ultimately surfaced the Newton–Schulz approach.

5 Acknowledgments

We thank Verda for providing continued access to B200 GPUs. We also thank the maintainers of the Flash Linear Attention and FlashInfer repositories for providing excellent learning resources for GPU programming.

Appendix

A Metadata kernel details

The chunk-parallel kernels (`kkt_inv_uw`, `O`) iterate over chunks in a chunk-parallel manner: each CTA picks a global chunk ID and computes the chunk independently. The `H` kernel, by contrast, must walk chunks sequentially within each sequence because of the state recurrence, and parallelizes over V -splits and heads instead. To support both access patterns on a packed variable-length batch, the metadata kernel emits two arrays:

- `chunk_offsets` $[N + 1]$ (`int32`): a prefix sum of per-sequence chunk counts. Entry i gives the global chunk index where sequence i starts; entry N is the total chunk count C_{tot} . The `H` kernel uses `chunk_offsets` $[i]$ to find the chunk slice of sequence i in the chunk-indexed arrays U , W , and H .
- `chunk_indices` $[C_{\text{tot}}, 2]$ (`int32`): a flat list of (`seq_id`, `chunk_id`) pairs, one per chunk. Each chunk-parallel CTA reads its tuple at index `global_chunk_id` and uses it to look up the input slice `cu_seqlens` $[\text{seq_id}] + \text{chunk_id} * \text{BT}$. This inverse lookup turns flat chunk parallelism over a packed batch into per-chunk addressing.

For example, for $N = 2$ sequences with $(3, 5)$ chunks:

$$\text{chunk_offsets} = [0, 3, 8], \quad \text{chunk_indices} = [(0, 0), (0, 1), (0, 2), (1, 0), (1, 1), (1, 2), (1, 3), (1, 4)].$$

The two arrays are inverses of each other: `chunk_indices` is the forward map `global_chunk_id` $\rightarrow$ (`seq_id`, `chunk_id`), while `chunk_offsets` is the inverse map `seq_id` $\rightarrow$ `global_chunk_id` base. Pre-materializing both lets each downstream kernel do an $O(1)$ lookup instead of an $O(N)$ scan at runtime.

Implementation. `prep_meta_v2` runs in a single CTA with $\lceil N/32 \rceil$ warps. Each thread handles one sequence: it loads two consecutive `cu_seqlens` entries, computes

$$\text{num_chunks} = \left\lceil \frac{\text{cu_seqlens}[i+1] - \text{cu_seqlens}[i]}{B_T} \right\rceil,$$

runs a Kogge–Stone warp prefix scan ($\log_2 32 = 5$ shuffle steps) followed by a cross-warp combine in shared memory, and then writes its `chunk_indices` slice and `chunk_offsets` entry. The whole kernel runs entirely on the device, eliminating the host round-trip that FLA’s `prepare_chunk_indices` incurs.

B Equations for GDN chunk implementation

B.1 kkt, inverse, and UW kernel

For one chunk and one V -head, given $K \in \mathbb{R}^{B_T \times K}$, $V \in \mathbb{R}^{B_T \times V}$, the per-token gate inputs $a, b \in \mathbb{R}^{B_T}$, and the per-head scalars A_{log} and `dt_bias`, the kernel computes the cumulative gate $g_{\text{cu}} \in \mathbb{R}^{B_T}$ and the outputs $U \in \mathbb{R}^{B_T \times V}$, $W \in \mathbb{R}^{B_T \times K}$ via:

$$\begin{aligned} \beta_t &= \sigma(b_t), \\ g_t &= -\exp(A_{\text{log}}) \text{softplus}(a_t + \text{dt_bias}), \\ g_{\text{cu}}[t] &= \sum_{s \leq t} g_s, \\ A_{ij} &= \begin{cases} (KK^\top)_{ij} \beta_i \exp(g_{\text{cu}}[i] - g_{\text{cu}}[j]) & i > j, \\ 0 & \text{otherwise,} \end{cases} \\ U &= (I + A)^{-1} \text{diag}(\beta) V, \\ W &= (I + A)^{-1} \text{diag}(\beta \odot \exp(g_{\text{cu}})) K. \end{aligned}$$

B.2 H kernel

$$\begin{aligned}
V_{new} &= U - WH^\top, \\
\tilde{V}_{new} &= V_{new} \text{diag}(\exp(g_{\text{cu}}[t_{\text{end}}] - g_{\text{cu}}[t])), \\
\tilde{H} &= \exp(g_{\text{cu}}[t_{\text{end}}]) H, \\
H_{new} &= \tilde{H} + \tilde{V}_{new}^\top K.
\end{aligned}$$

C Matrix inversion

Neumann series. Given a 16×16 strictly lower triangular matrix A , we have the following identity:

$$(I + A)^{-1} = (I - A)(I + A^2)(I + A^4)(I + A^8). \quad (1)$$

Proof. Since A is strictly lower triangular of size 16×16 , it is nilpotent and satisfies $A^{16} = 0$. Therefore, the Taylor/Neumann expansion of $(I + A)^{-1}$ terminates:

$$(I + A)^{-1} = \sum_{k=0}^{15} (-A)^k = I - A + A^2 - \dots - A^{15}.$$

On the other hand, by the polynomial identity

$$(1 - x)(1 + x^2)(1 + x^4)(1 + x^8) = 1 - x + x^2 - \dots - x^{15},$$

we obtain

$$(I - A)(I + A^2)(I + A^4)(I + A^8) = I - A + A^2 - \dots - A^{15}.$$

Hence,

$$(I + A)^{-1} = (I - A)(I + A^2)(I + A^4)(I + A^8).$$

Newton–Schulz iteration. Given an estimate X_k of M^{-1} , we can refine the estimate with:

$$X_{k+1} = 2X_k - X_k M X_k. \quad (2)$$

Proof. Assume the initial estimate has certain errors that can be represented as $X_k M = I + E_k$. Substituting this into the iterative equation:

$$X_{k+1} M = 2X_k M - X_k M X_k M \quad (3)$$

$$= 2(I + E_k) - (I + E_k)^2 \quad (4)$$

$$= I - E_k^2 \quad (5)$$

The new error becomes $E_{k+1} = -E_k^2$. If $\|E_k\| < 1$, then the error shrinks quadratically. We can use the first few terms of the Neumann series as the initial guess for Newton–Schulz iteration. Using only two terms, $X_0 = I - A$, also works in practice.

D H kernel warp specialization

Here are extra details about the H kernel.

MMA	H warpgroup	V warpgroup
$V_{new}^\top = U^\top - HW^\top$	Load FP32 H from tmem, convert to BF16, store to tmem (MMA input).	Load BF16 U from smem, convert to FP32, store to tmem (MMA accumulator).
$H_{new} = \tilde{H} + \tilde{V}_{new}^\top K$	Load FP32 H from tmem, apply scaling, store to tmem (MMA accumulator).	Load FP32 $V_{new}^\top$ from tmem, apply scaling, convert to BF16, store to tmem (MMA input).

Table 3: Symmetric workload between H and V warps.

Warps (Role)	Description
Warp 9 (TMA)	Load W , V , and K from gmem to smem using TMA.
Warps 4-5 (CUDA H)	Load H (from h_0 on first chunk, otherwise from tmem), convert FP32 $\rightarrow$ BF16, write to tmem as input for MMA #1 and to gmem for the O kernel.
Warps 2-3 (CUDA V)	Load U from smem, convert BF16 $\rightarrow$ FP32, write to tmem as accumulator for MMA #1.
Warp 8 (MMA #1)	Wait on H (tmem), U (tmem), and W (smem), then issue MMA: $V_{new}^\top = U^\top - HW^\top$.
Warps 4-5 (CUDA H)	Load $g_{cu, last}$, compute scaling, and produce $\tilde{H} = H \cdot \exp(g_{cu, last})$ in tmem as accumulator for MMA #2.
Warps 2-3 (CUDA V)	Load g_{cu} , compute scaling, read V_{new} from tmem, store V_{new} to gmem, and produce $\tilde{V}_{new}$ in tmem as input for MMA #2.
Warp 8 (MMA #2)	Wait on $\tilde{H}$ (tmem), $\tilde{V}_{new}$ (tmem), and K (smem), then issue MMA: $H_{new} = \tilde{H} + \tilde{V}_{new}^\top K$.

Table 4: Warp-specialized pipeline of the H kernel. CUDA warps prepare Tensor Memory operands, while the MMA warp issues two dependent Tensor Core operations ($HW^\top$ and $\tilde{V}_{new}^\top K$) per chunk.

E Output kernel details

Centered gate factorization. For each output head in a chunk, the CUDA warps center the cumulative gates via

$$c = \frac{\min(g) + \max(g)}{2}.$$

They then compute

$$g_i^+ = \exp(g_i - c), \quad g_j^- = \exp(c - g_j), \quad \alpha = \exp(c).$$

This gives

$$\exp(g_i - g_j) = g_i^+ g_j^-, \quad \exp(g_i) = g_i^+ \alpha.$$

The Tensor Memory attention tile stores only the lower-triangular, column-scaled part

$$\text{ATTN}_{ij} = \mathbf{1}_{j \leq i} (q_i \cdot k_j) g_j^-.$$

The row factor g_i^+ , α , and the global scale are applied in the register epilogue:

$$O_i = \text{scale } g_i^+ \left(\alpha (QS^\top)_i + \sum_{j \leq i} (q_i \cdot k_j) g_j^- V_j' \right).$$

This matches the CUDA epilogue $\text{scale } g_i^+ (0V + \alpha QH)$ and avoids materializing the fully gate-scaled attention matrix.

TMEM columns	Name	Use
0–63	QK	Raw $QK^\top$ tile.
64–127	ATTN0	Lower-triangular attention tile for head 0.
128–191	ATTN1	Lower-triangular attention tile for head 1.
192–319	OUT	Shared output accumulator for the two heads.
320–447	QH0	$QH = QS^\top$ tile for head 0.
0–127	QH1	Aliased QH tile for head 1 after QK and ATTN0 are dead.

Table 5: Tensor Memory layout in the output kernel.

Tensor Memory layout. The kernel allocates 512 Tensor Memory columns. The live ranges are organized as shown in Table 5.

The alias is required by capacity. Even with OUT shared between the two output heads, a non-aliased QH1 would require another 128 columns, bringing the layout to

$$64_{\text{QK}} + 64_{\text{ATTN0}} + 64_{\text{ATTN1}} + 128_{\text{OUT}} + 128_{\text{QH0}} + 128_{\text{QH1}} = 576$$

columns, which exceeds the 512-column Tensor Memory budget. Thus the kernel must both serialize reuse of OUT across the two heads and alias QH1 onto the [QK | ATTN0] region.

Pipeline and release rules. The TMA warp first fills the double-buffered Q/K stages and the per-head V'/S stages. After the MMA warp commits QK, CUDA reads QK into registers and releases the QK region. CUDA then writes ATTN0 and ATTN1 from the resident QK registers. For chunks after the first one, both the next QK rebuild and the next ATTN0 materialization wait on `output_tmem_release_barrier`, because the next chunk’s [QK | ATTN0] region overlaps the previous chunk’s QH1 alias.

Within a chunk, head 1’s QH tile may overwrite [QK | ATTN0] only after CUDA has released QK and after head 0’s QH and OV MMA commits are complete. Once QH1 is built, the Q/K shared-memory stage and the head-1 S stage can be returned to the TMA producer. Because both heads share OUT, head 1’s OV computation waits until CUDA has drained head 0’s OUT and QH0. The final head-1 drain releases `output_tmem_release_barrier`, making the aliased Tensor Memory region available to the next chunk stream iteration.

Store epilogue. CUDA drains OUT and QH from Tensor Memory into registers, computes

$$\text{scale } g_i^+ (\text{OV} + \alpha \text{QH}),$$

converts pairs of outputs to `bf16x2`, and stores with no-allocate global stores. Partial chunks predicate the row stores, while full chunks use the unconditional fast path. Tensor Memory can be released before the global stores complete because the stores use only register-resident values.